УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 16

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ ДОСТУПА И ИЗВЛЕЧЕНИЯ ДАННЫХ ИЗ СЕТИ ИНТЕРНЕТ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для доступа и извлечения данных из сети Интернет

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Особенности отправки данных по протоколу HTTP

Модуль **http.client** позволяет получить информацию из Интернета по протоколам **HTTP** и **HTTPS**. Отправить запрос можно методами **GET**, **POST** и **HEAD**.

HTTPConnection(<Домен>[, <Порт>[, timeout[, source_address]]])

В первом параметре указывается название домена без протокола. Во втором параметре задается номер порта - если параметр не указан, используется порт 80. Номер порта можно также задать после названия домена через двоеточие.

Пример создания объекта соединения:

```
from http.client import HTTPConnection
con = HTTPConnection("test1.ru")
con2 = HTTPConnection("test1.ru", 80)
con3 = HTTPConnection("test1.ru:80")
```

После создания объекта соединения необходимо отправить запрос, возможно, с параметрами, вызвав метод **request()** класса **HTTPConnection**. Формат метода:

request (<Метод>, <Путь>[, body=None][, headers=<Заголовки>])

В первом параметре указывается метод передачи данных (**GET**, **POST** или **HEAD**). Второй параметр задает путь к запрашиваемому файлу или вызываемой программе, отсчитанный от корня сайта. Если для передачи данных используется метод **GET**, то после вопросительного знака можно указать передаваемые данные. В необязательном третьем параметре задаются данные, которые передаются методом **POST**, - допустимо указать строку, файловый объект или последовательность. Четвертый параметр задает в виде словаря **HTTP**-заголовки, отправляемые на сервер.

Получить объект результата запроса позволяет метод **getresponse()**. Он возвращает результат выполненного запроса, представленный в виде объекта класса **HTTPResponse**. Из него можно получить ответ сервера.

Прочитать ответ сервера (без заголовков) можно с помощью метода **read([<Количество байт>])** класса **HTTPResponse**. Если параметр не указан, метод **read()** возвращает все данные, а при наличии параметра - только указанное количество байтов при каждом вызове. Если данных больше нет, метод возвращает пустую строку. Прежде чем выполнять другой запрос, данные должны быть получены полностью. Метод **read()** возвращает последовательность байтов, а не строку. Закрыть объект соединения позволяет метод **close()** класса **HTTPConnection**. Для примера отправим запрос методом **GET** и прочитаем результат.

```
from http.client import HTTPConnection
from urllib.parse import urlencode
data = urlencode({"color": "Красный", "var": 15}, encoding="cp1251")
headers = { "User-Agent": "MySpider/1.0",
"Accept": "text/html, text/plain, application/xml",
"Accept-Language": "ru, ru-RU",
"Accept-Charset": "windows-1251",
"Referer": "/index.php" }
con = HTTPConnection("mail.ru")
con.request("GET", "/testrobots.php?%s" % data, headers=headers)
result  = con.getresponse() #  Создаем объект результата
print (result.read().decode("cp1251" )) # Читаем данные
<html> <head><title>301 Moved Permanently</title></head>
<body bgcolor="white">
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.14.1</center>
</body>
</html>
con.close() # Закрываем объект соединения
```

Теперь отправим данные методом **POST**. В этом случае в первом параметре метода **request()** задается значение **"POST"**, а данные передаются через третий параметр. Размер строки запроса автоматически указывается в заголовке **Content-Length**. Пример отправки данных методом **POST** приведен ниже.

```
from http.client import HTTPConnection
from urllib.parse import urlencode
data = urlencode({"color": "Красный", "var": 15}, encoding="cp1251")
headers = { "User-Agent": "MySpider/1.0",
"Accept": "text/html, text/plain, application/xml",
"Accept-Language": "ru, ru-RU",
"Accept-Charset": "windows-1251",
"Content-Type": "application/x-www-form-urlencoded",
"Referer": "/index.php" }
con = HTTPConnection("mail.ru")
```

```
con.request("POST", "/testrobots.php?%s" % data, headers=headers)
result  = con.getresponse() #  Создаем объект результата
print (result.read().decode("cp1251" )) # Читаем данные
<html> <head><title>301 Moved Permanently</title></head>
<body bgcolor="white">
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.14.1</center>
</body>
</html>
con.close() # Закрываем объект соединения
```

Обратите внимание на заголовок **Content-Type**. Если в нем указано значение **application/x-www-form-urlencoded**, это означает, что отправлены данные формы. При наличии этого заголовка некоторые программные платформы автоматически производят разбор строки запроса. Например, в **PHP** переданные данные будут доступны через глобальный массив **$_POST**. Если же заголовок не указать, то данные через массив **$_POST** доступны не будут.

4.2 Класс HTTPResponse

Класс **HTTPResponse**, представляющий результат запроса, предоставляет следующие методы и атрибуты:

– **getheader (<Заголовок>[, <Значение по умолчанию>])** - возвращает значение указанного заголовка. Если заголовок не найден, возвращается значение **None** или значение из второго параметра.

Пример:

```
result.getheader ("Content-Type")
'text/html'
print(result.getheader("Content-Types"))
None
result.getheader("Content-Types", 10)
10
```

– **getheaders ()** – возвращает все заголовки ответа сервера в виде списка кортежей. Каждый кортеж состоит из двух элементов: **(<Заголовок>, <Значение>)**. Пример получения заголовков ответа сервера:

```
result.getheaders()
[('Server', 'nginx/1.14.1'), ('Date', 'Thu, 03 Jan 2019 13:42:22 GMT'),
('Content-Type', 'text/html'), ('Content-Length', '185'),
('Connection', 'keep-alive'), ('Location', 'https://mail.ru'),
('X-XSS-Protection', '1; mode=block;
report=https://cspreport.mail.ru/xxssprotection'),
('X-Content-Type-Options', 'nosniff')]
```

С помощью функции **dict()** такой список можно преобразовать в словарь:

```
dict(result.getheaders())
{'Server': 'nginx/1.14.1', 'Content-Length': '185',
'Location': 'https://mail.ru', 'Connection': 'keep-alive',
```

'X-XSS-Protection': '1; mode=block;
report=https://cspreport.mail.ru/xxssprotection',
'Date': 'Thu, 03 Jan 2019 13:42:22 GMT',
'Content-Type': 'text/html', 'X-Content-Type-Options': 'nosniff'}

– **status** – код возврата в виде числа. Успешными считаются коды от 200 до 299 и код 304, означающий, что документ не был изменен со времени последнего посещения. Коды 301 и 302 задают перенаправление. Код 401 означает необходимость авторизации, 403 - доступ закрыт, 404 - документ не найден, а код 500 и коды выше информируют об ошибке сервера.

Пример:

result.status
301

– **reason** – текстовый статус возврата:

result.reason # При коде 200
'OK'
result.reason # При коде 301
'Moved Permanently'

– **version** – версия протокола в виде числа (число 10 для протокола **HTTP/1.0** и число 11 для протокола **HTTP/1.1**).

Пример:

result.version # Протокол HTTP/1.1
11

– **msg** – объект **http.client.HTTPMessage**. С его помощью можно получить дополнительную информацию о заголовках ответа сервера. Если объект передать функции **print()**, то можно получить все заголовки ответа сервера.

Пример:

print (result.msg)
Server: nginx/1.14.1
Date: Thu, 03 Jan 2019 13:42:22 GMT
Content-Type: text/html
Content-Length: 185
Connection: keep-alive
Location: https://mail.ru
X-XSS-Protection: 1; mode=block;
report=https://cspreport.mail.ru/xxssprotection
X-Content-Type-Options: nosniff

4.3 Объект http.client.HTTPMessage

Рассмотрим основные методы и атрибуты объекта **http.client.HTTPMessage**:

– **as_string ([unixfrom=False] [, maxheaderlen=0])** – возвращает все заголовки ответа сервера в виде строки:

result.msg.as_string()
'Server: nginx/1.14.1\nDate: Thu, 03 Jan 2019 13:42:22 GMT\n

```
Content-Type: text/html\n
Content-Length: 185\n
Connection: keep-alive\n
Location: https://mail.ru\n
X-XSS-Protection: 1; mode=block;
report=https://cspreport.mail.ru/xxssprotection\n
X-Content-Type-Options: nosniff\n\n'
```

- **items ()** – список всех заголовков ответа сервера:

```
result.msg.items()
[('Server', 'nginx/1.14.1'),
('Date', 'Thu, 03 Jan 2019 13:42:22 GMT'), ('Content-Type', 'text/html'),
('Content-Length', '185'), ('Connection', 'keep-alive'),
('Location', 'https://mail.ru'), ('X-XSS-Protection', '1;
mode=block; report=https://cspreport.mail.ru/xxssprotection'),
('X-Content-Type-Options', 'nosniff')]
```

- **keys ()** – список ключей в заголовках ответа сервера:

```
>>> result.msg.keys()
['Server', 'Date', 'Content-Type', 'Content-Length',
'Connection', 'Location', 'X-XSS-Protection', 'X-Content-Type-Options']
```

- **values ()** – список значений в заголовках ответа сервера:

```
result.msg.values()
['nginx/1.14.1', 'Thu, 03 Jan 2019 13:42:22 GMT', 'text/html',
'185', 'keep-alive', 'https://mail.ru', '1; mode=block;
report=https://cspreport.mail.ru/xxssprotection', 'nosniff']
```

- **get (<Заголовок>[, failobj=None])** – возвращает значение указанного заголовка в виде строки. Если заголовок не найден, возвращается **None** или значение из второго параметра.

Примеры:

```
print(result.msg.get("Server"))
nginx/1.14.1
print(result.msg.get("X-Powered-By"))
None
print(result.msg.get("X-Powered-By", failobj=10))
10
```

- **get_all (<Заголовок>[, failobj=None])** – возвращает список всех значений указанного заголовка. Если заголовок не найден, возвращается **None** или значение из второго параметра.

Пример:

```
print(result.msg.get_all("Server"))
['nginx/1.14.1']
```

- **get_content_type ()** – возвращает **MIME**-тип документа из заголовка **Content-Type**:

```
print(result.msg.get_content_type())
text/html
```

– **get_content_maintype ()** – возвращает первую составляющую **MIME**-типа:

```
print(result.msg.get_content_maintype())
text
```

– **get_content_subtype ()** – возвращает вторую составляющую **MIME**-типа:

```
print(result.msg.get_content_subtype())
html
```

– **get_content_charset ([failobj=None])** – позволяет получить кодировку из заголовка **Content-Type**. Если кодировка не найдена, возвращается **None** или значение из параметра **failobj**. Получим кодировку документа:

```
result.msg.get_content_charset ()
'windows-1251'
```

Для примера отправим запрос методом **HEAD** и выведем заголовки ответа сервера.

```
from http.client import HTTPConnection
headers = { "User-Agent": "MySpider/1.0",
"Accept": "text/html, text/plain, application/xml",
"Accept-Language": "ru, ru-RU",
"Accept-Charset": "windows-1251",
"Referer": "/index.php" }
con = HTTPConnection("mail.ru")
con.request("HEAD", "/testrobots.php?%s" % data, headers=headers)
result  = con.getresponse() #  Создаем объект результата
print (result.msg)
Server: nginx/1.14.1
Date: Thu, 03 Jan 2019 14:40:17 GMT
Content-Type: text/html
Content-Length: 185
Connection: keep-alive
Location: https://mail.ru
X-XSS-Protection: 1; mode=block;
report=https://csprepоrt.mail.ru/xxssprotection
X-Content-Type-Options: nosniff
result.read() # Данные не передаются, только заголовки!
b"
con.close()
```

Рассмотрим основные **HTTP**-заголовки и их предназначение:

– **GET** - заголовок запроса при передаче данных методом **GET**;

– **POST** - заголовок запроса при передаче данных методом **POST**;

– **Host** - название домена;

– **Accept** - **MIME**-типы, поддерживаемые **Web**-браузером;

– **Accept-Language** - список поддерживаемых языков в порядке предпочтения;

– **Accept-Charset** - список поддерживаемых кодировок;

– **Accept-Encoding** - список поддерживаемых методов сжатия;

- **Content-Type** - тип передаваемых данных;
- **Content-Length** - длина передаваемых данных при методе **POST**;
- **Cookie** - информация об установленных **cookies**;
- **Last-Modified** - дата последней модификации файла;
- **Location** - перенаправление на другой **URL**-адрес;
- **Pragma** - заголовок, запрещающий кэширование документа в протоколе **HTTP/1.0**;
- **Cache-Control** - заголовок, управляющий кэшированием документа в протоколе **HTTP/1.1**;
- **Referer** - **URL**-адрес, с которого пользователь перешел на наш сайт;
- **Server** - название и версия программного обеспечения **Web**-сервера;
- **User-Agent** - информация об используемом **Web**-браузере.

Получить полное описание заголовков можно в спецификации **RFC 2616**, расположенной по адресу **http://www.w3.org/Protocols/rfc2616/rfc2616-sec14.html**. Чтобы "подсмотреть" заголовки, отправляемые **Web**-браузером и сервером, можно воспользоваться модулем **Firebug** для **Firefox**. Для этого на вкладке **Сеть** следует щелкнуть мышью на строке запроса. Кроме того, можно установить панель **ieHTTPHeaders** в **Web**-браузере **Internet Explorer**.

4.4 Пример решения задачи на языке программирования Python

Задача. Напишите программу на Python, которая отправляет HTTP-GET запрос к заданному URL-адресу, получает ответ от сервера и выводит код состояния (status code). Также программа должна сообщать, успешно ли выполнен запрос (код 200 означает успешное выполнение).

Решение задачи:

```
import requests
url = input("Введите URL: ")
response = requests.get(url)
print("Код состояния:", response.status_code)
if response.status_code == 200:
    print("Запрос выполнен успешно")
else:
    print("Ошибка при выполнении запроса")
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите программу на Python, которая отправляет HTTP-GET запрос к заданному веб-адресу и выводит код состояния ответа сервера.

2 Напишите программу, которая загружает веб-страницу по указанному URL и выводит первые 200 символов полученного HTML-документа.

3 Напишите программу, которая определяет, доступен ли веб-ресурс, и выводит сообщение «Ресурс доступен» или «Ресурс недоступен».

4 Напишите программу, которая получает содержимое веб-страницы и выводит её общий размер в байтах.

5 Напишите программу, которая запрашивает URL у пользователя, отправляет HTTP-запрос и выводит заголовки ответа сервера.

6 Напишите программу, которая проверяет, использует ли указанный веб-ресурс защищённый протокол HTTPS.

7 Напишите программу, которая загружает HTML-страницу и подсчитывает количество символов в тексте ответа.

8 Напишите программу, которая отправляет запрос к веб-ресурсу и определяет тип содержимого (Content-Type).

9 Напишите е программу, которая выводит имя сервера (Server) из HTTP-заголовков ответа.

10 Напишите программу, которая получает веб-страницу и сохраняет её HTML-код в текстовый файл.

11 Напишите программу, которая отправляет HTTP-запрос и выводит время отклика сервера в секундах.

12 Напишите программу, которая определяет кодировку полученного веб-документа.

13 Напишите программу, которая проверяет, был ли запрос успешно выполнен (код ответа 200), и выводит соответствующее сообщение.

14 Напишите программу, которая проверяет, содержит ли веб-страница заданное пользователем слово, и выводит количество его вхождений.

15 Напишите программу, которая извлекает все ссылки (href) из HTML-кода страницы и выводит их список.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите, какой формат имеет конструктор класса HTTPConnection в языке программирования Python.

7.2 Приведите формат метода request () в языке программирования Python.

7.3 Опишите методы и атрибуты, которые предоставляет класс HTTPResponse, представляющий результат запроса в языке программирования Python.

7.4 Перечислите основные HTTP-заголовки и охарактеризуйте их предназначение в языке программирования Python.

7.5 Охарактеризуйте основные методы и атрибуты объекта http.client.HTTPMessage в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.